

Introduction to Software Testing

Chapter 7.1

Engineering Criteria for Technologies

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

The Technologies

- Chapters 1-5 emphasize **criteria on four** models of software
- **Emphasis** in each chapter was first on the criteria, then on how to construct the models from different software artifacts
- This chapter discusses **how to apply** the criteria to specific technologies
 - Most of the ideas in this chapter were developed **after** the year **2000**
 - Thus they are still **evolving**

Chapter 7 Outline

1. **Object-Oriented Software**
2. **Web Applications and Web Services**
3. **Graphical User Interfaces**
4. **Real-Time and Embedded Software**

Section 7.1 Outline

1. Overview

2. Types of Object-Oriented Faults

1. Example
2. The Yo-Yo Graph and Polymorphism
3. Categories of Inheritance Faults
4. Testing Inheritance, Polymorphism and Dynamic Binding
5. Object-Oriented Testing Criteria

Inheritance

Allows **common features** of many classes to be defined in one class

A **derived** class has everything its parent has, plus it can:

- **Enhance** derived features (overriding)
- **Restrict** derived features
- **Add** new features (extension)

Inheritance (2)

Declared type: The type given when an object reference is declared

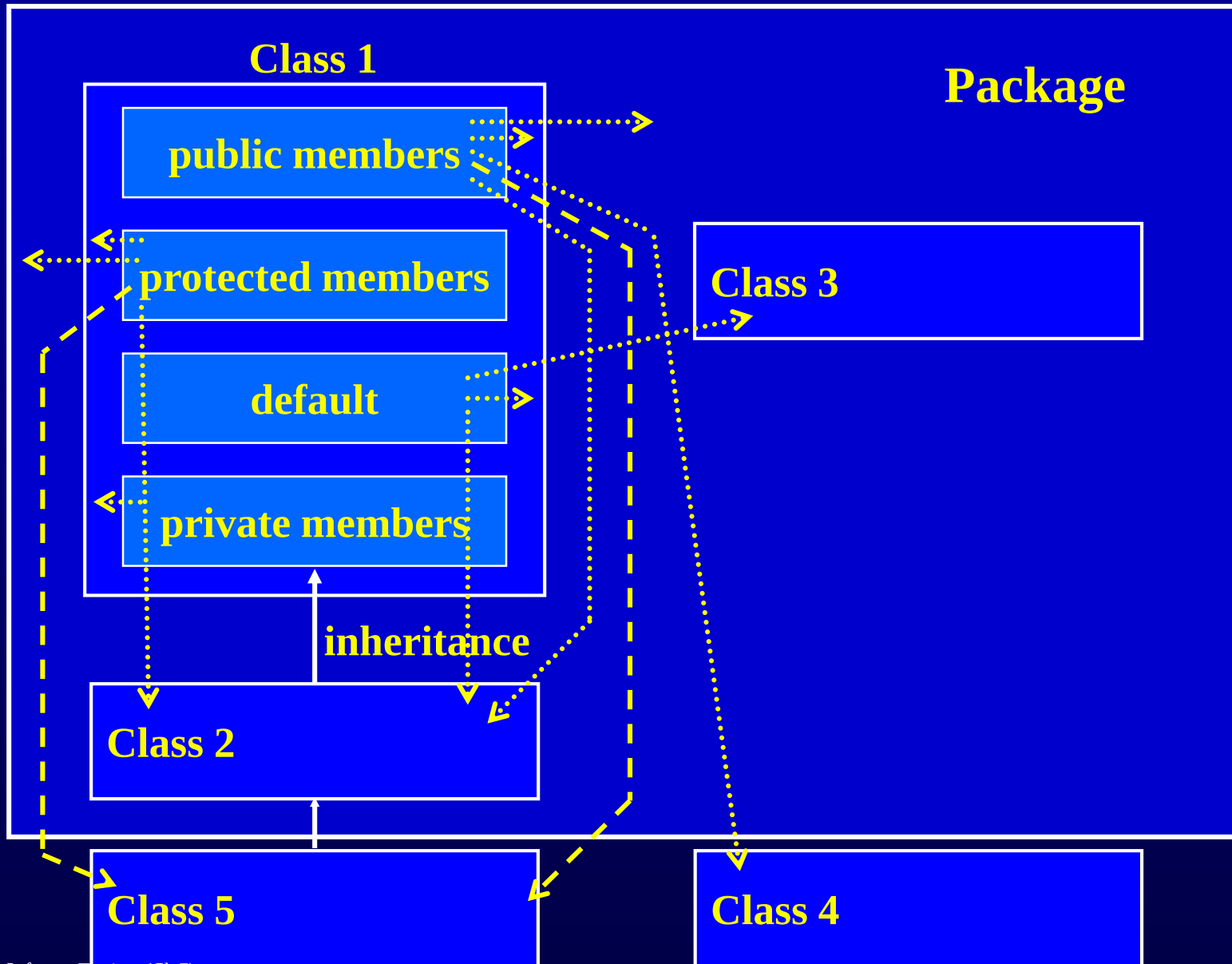
`Clock w1; // declared type Clock`

Actual type: The type of the current object
`w1 = new Watch(); // actual type Watch`

In Java, the method that is executed is the lowest version of the method defined between the actual and declared types in the inheritance hierarchy



Access Control (in Java)



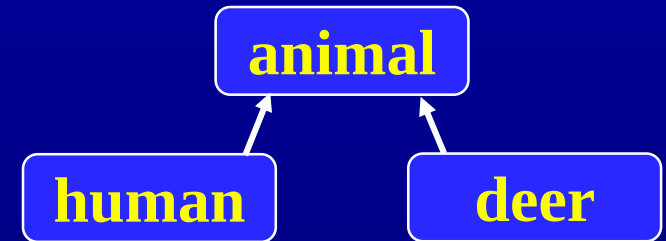
Polymorphism

- The same variable can have **different types** depending on the program execution
- If *B* inherits from *A*, then an object of type *B* **can be used** when an object of type *A* is expected
- If both *A* and *B* **define the same method *M*** (*B* overrides *A*), then the same statement might call either *A*'s version of *M* or *B*'s version

Subtype and Subclass Inheritance

- **Subtype Inheritance** : If B inherits from A, any object of type B can be substituted for an object of type A

- A *laptop* “is a” special type of *computer*
- Called *substitutability*



- **Subclass Inheritance** : Objects of type B may not be substituted for objects of type A

- Objects of B may not be “*type compatible*”
- In Java’s collection framework, a *Stack* inherits from a *Vector* ... convenient for implementation, but a stack is definitely **not** a vector



Testing OO Software

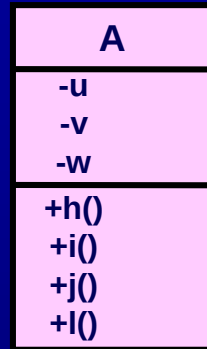
1. **Intra-method testing** : Testing individual methods within classes
2. **Inter-method testing** : Multiple methods within a class are tested in concert
3. **Intra-class testing** : Testing a single class, usually using sequences of calls to methods within the class
4. **Inter-class testing** : More than one class is tested at the same time (integration)

Section 7.1 Outline

1. Overview
2. Types of Object-Oriented Faults
 1. **Example**
 2. The Yo-Yo Graph and Polymorphism
 3. Categories of Inheritance Faults
 4. Testing Inheritance, Polymorphism and Dynamic Binding
 5. Object-Oriented Testing Criteria

Example DU Pairs and Anomalies

Consider what happens when an overriding method has a different def-set than the overridden method



Method	Defs	Uses
A::h ()	{A::u, A::w}	
A::i ()		{A::u}
A::j ()	{A::v}	{A::w}
A::l ()		{A::v}
B::h ()	{B::x}	
B::i ()		{B::x}
C::i ()		
C::j ()	{C::y}	{C::y}

A::h() calls j(),
B::h() does not

def-use

DU
anomaly

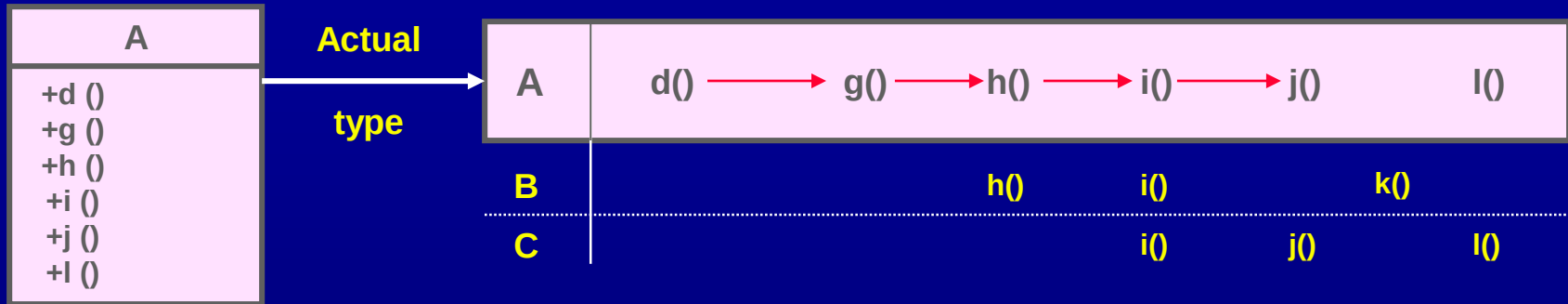
def-use

DU
anomaly

Section 7.1 Outline

1. Overview
2. Types of Object-Oriented Faults
 1. Example
 2. **The Yo-Yo Graph and Polymorphism**
 3. Categories of Inheritance Faults
 4. Testing Inheritance, Polymorphism and Dynamic Binding
 5. Object-Oriented Testing Criteria

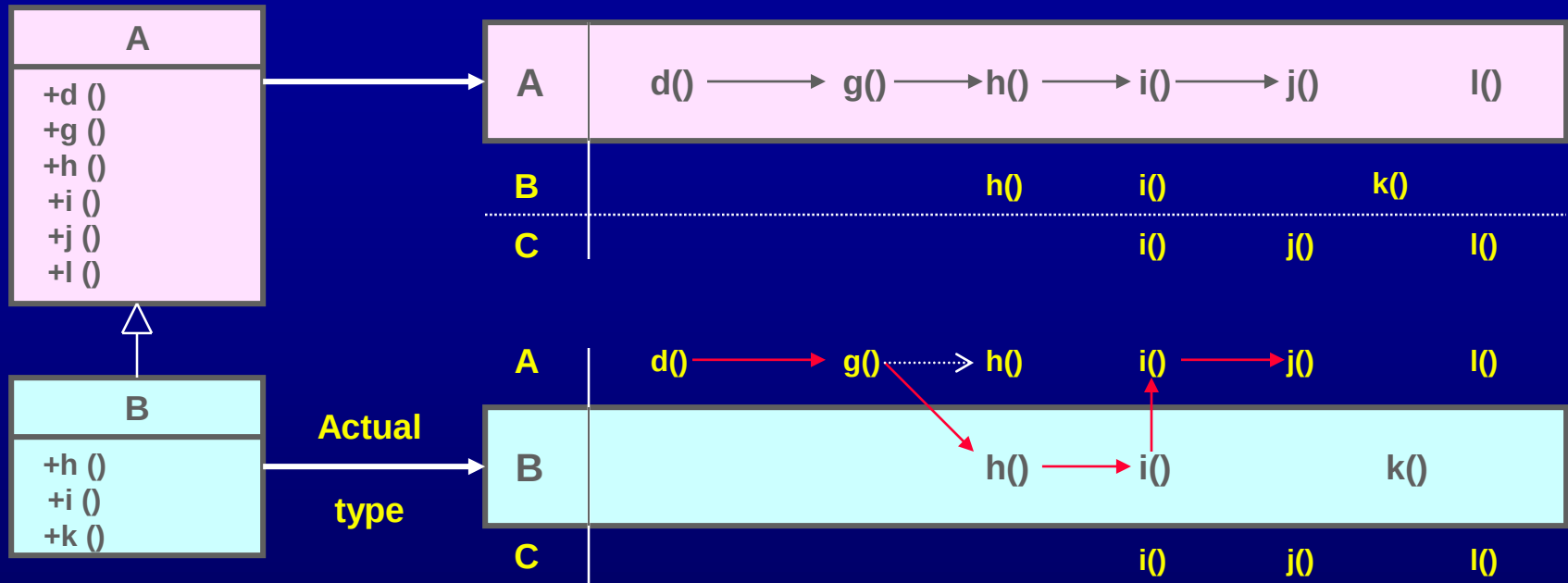
Polymorphism Headaches (Yo-Yo)



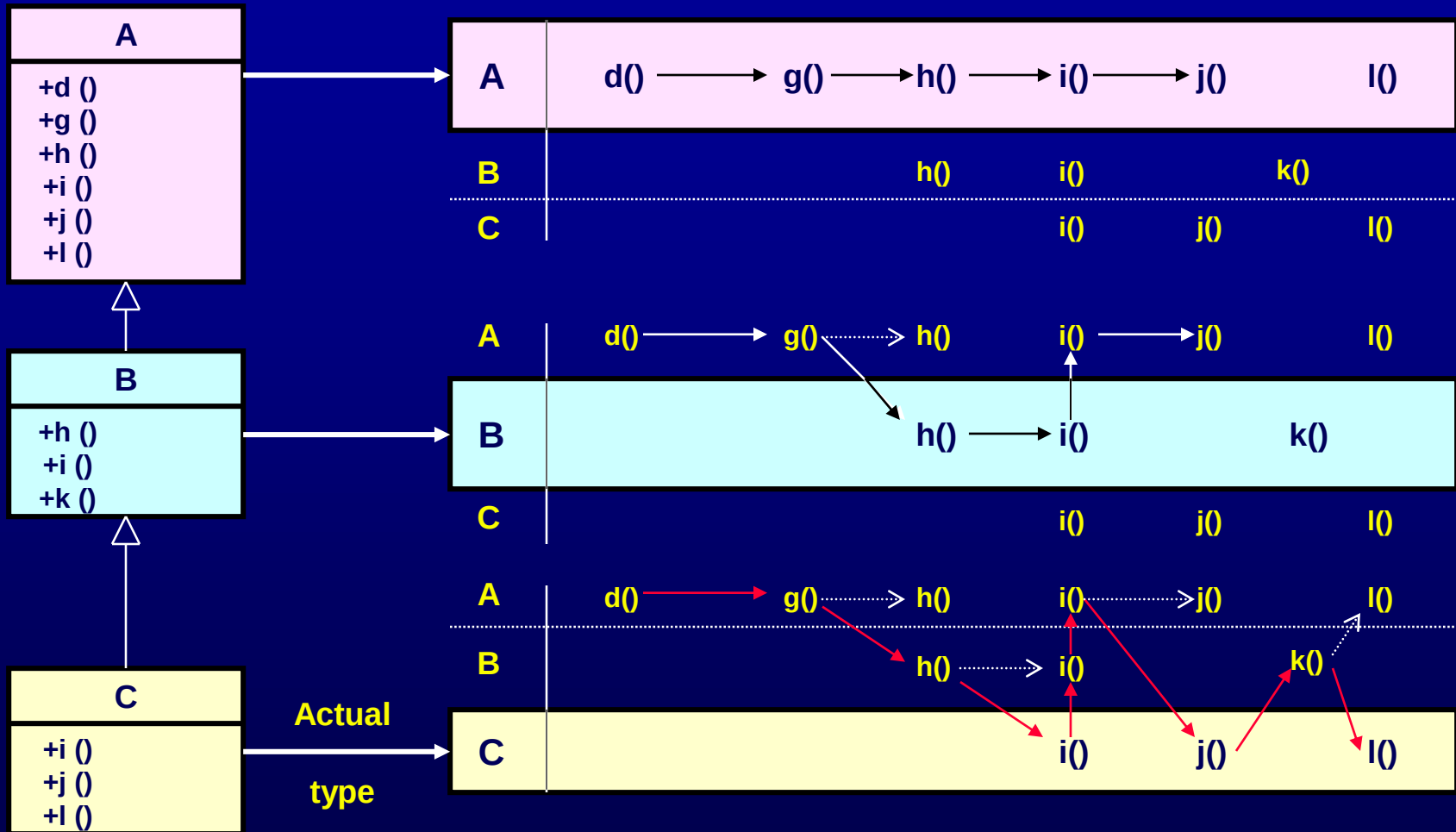
Object is of actual type A
A::d ()

**d() calls g(), which calls h(), which
 calls i(), which calls j()**

Polymorphism Headaches (Yo-Yo)



Polymorphism Headaches (Yo-Yo)



Object is of actual type C, C::d ()

Section 7.1 Outline

1. Overview
2. Types of Object-Oriented Faults
 1. Example
 2. The Yo-Yo Graph and Polymorphism
 3. **Categories of Inheritance Faults**
 4. Testing Inheritance, Polymorphism and Dynamic Binding
 5. Object-Oriented Testing Criteria

Potential for Faults in OO Programs

- Complexity is relocated to the connections among components
- Less static determinism – many faults can now only be detected at runtime
- Inheritance and Polymorphism yield vertical and dynamic integration
- Aggregation and use relationships are more complex
- Designers do not carefully consider visibility of data and methods

Object-oriented Faults

- Only consider **faults** that arise as a direct result of OO language features:
 - inheritance
 - polymorphism
 - constructors
 - visibility
- **Language independent** (as much as possible)

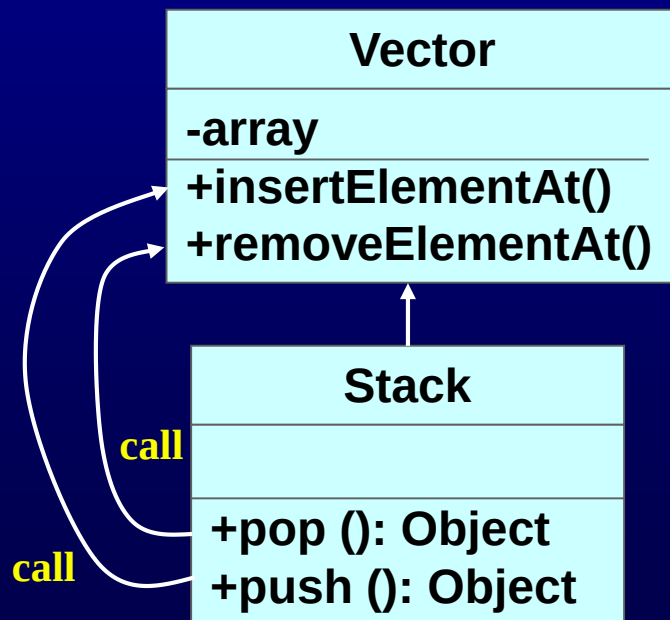
OO Faults and Anomalies

Acronym	Fault / Anomaly
ITU	Inconsistent Type Use
SDA	State Definition Anomaly
SDIH	State Definition Inconsistency
SDI	State Defined Incorrectly
IISD	Indirect Inconsistent State Definition
ACB1	Anomalous Construction Behavior (1)
ACB2	Anomalous Construction Behavior (2)
IC	Incomplete Construction
SVA	State Visibility Anomaly

**Examples
shown**

Inconsistent Type Use (ITU)

- No **overriding** (no polymorphism)
- *C* extends *T*, and *C* adds new methods (**extension**)
- An **object is used** “as a *C*”, then as a *T*, then as a *C*
- Methods in *T* can put object in state that is **inconsistent** for *C*

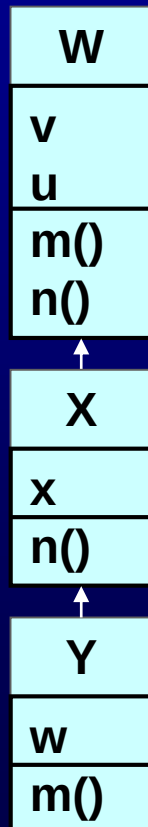


```
s.push ("Steffi");
s.push ("Joyce");
s.push ("Andrew");
dumb (s);
s.pop();
s.pop();
s.pop(); // Stack is empty!
```

```
void dumb (Vector v)
{
    v.removeElementAt (v.size()-1);
}
```

State Definition Anomaly (SDA)

- X extends W , and X overrides some methods
- The overriding methods in X fail to define some variables that the overridden methods in W defined

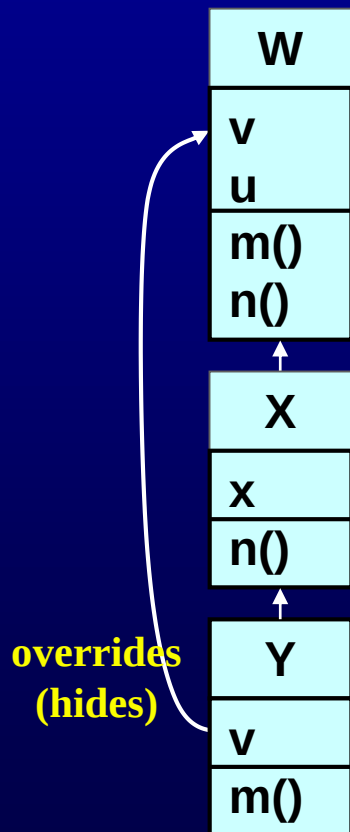


- $W::m()$ defines v and $W::n()$ uses v
- $X::n()$ uses v
- $Y::m()$ does not define v

For an object of actual type Y , a data flow anomaly exists and results in a fault if $m()$ is called, then $n()$

State Definition Inconsistency (SDIH)

- **Hiding** a variable, possibly accidentally
- If the descendant's version of the variable is defined, the **ancestor's** version may not be

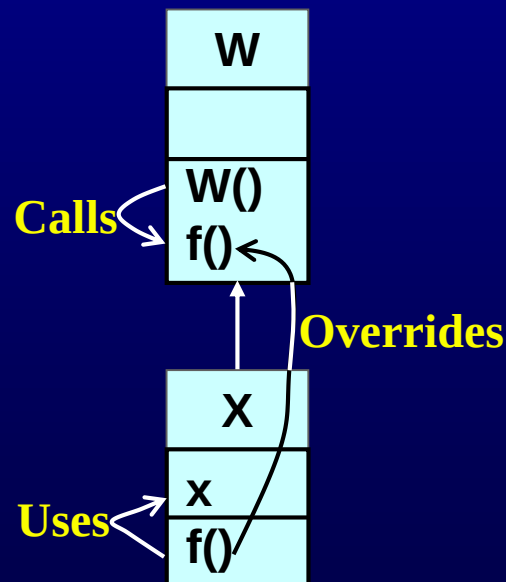


- *Y* overrides *W*'s version of *v*
- *Y::m()* defines *Y::v*
- *X::n()* uses *v* ... getting *W*'s version of *v*

For an object of actual type *Y*, a data flow inconsistency may exist and result in a fault if *m()* is called, then *n()*

Anomalous Construction Behavior (ACB1)

- **Constructor** of W calls a method $f()$
- A **child** of W , X , overrides $f()$
- $X::f()$ uses **variables** that should be defined by X 's constructor

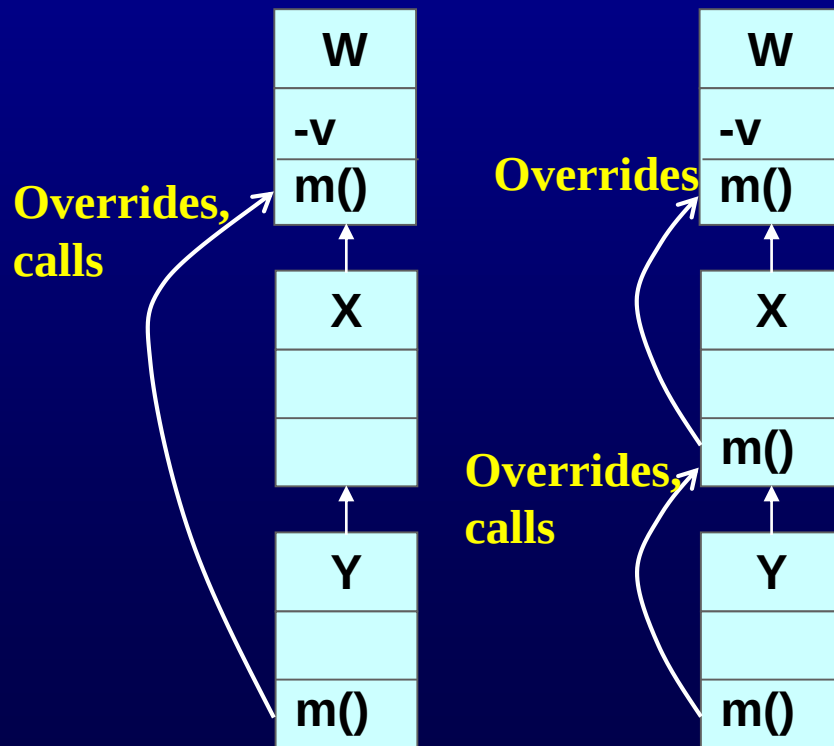


When an object of type X is constructed, $W()$ is run before $X()$.

When $W()$ calls $X::f()$, x is used, but has not yet been given a value!

State Visibility Anomaly (SVA)

- A private variable v is **declared** in ancestor W , and v is defined by $W::m()$
- X **extends** W and Y extends X
- Y overrides $m()$, and **calls** $W::m()$ to define v



$X::m()$ is added later

$Y::m()$ can no longer call $W::m()$!

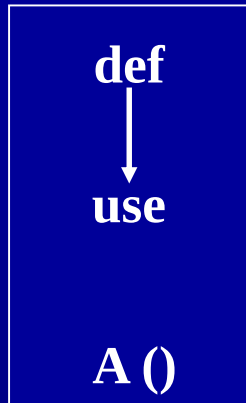
Section 7.1 Outline

1. Overview
2. Types of Object-Oriented Faults
 1. Example
 2. The Yo-Yo Graph and Polymorphism
 3. Categories of Inheritance Faults
 4. **Testing Inheritance, Polymorphism and Dynamic Binding**
 5. Object-Oriented Testing Criteria

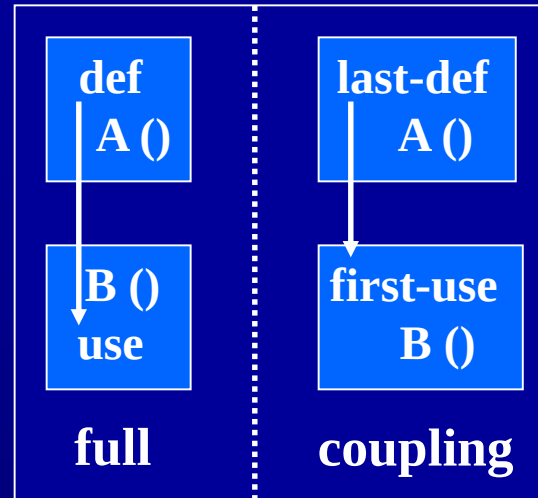
Coupling Sequences

- **Pairs** of method calls within body of method under test:
 - Made through a common instance context
 - With respect to a set of state variables that are commonly referenced by both methods
 - Consists of at least one coupling path between the two method calls with respect to a particular state variable
- Represent potential **state space interactions** between the called methods with respect to calling method
- Used to identify **points of integration** and testing requirements

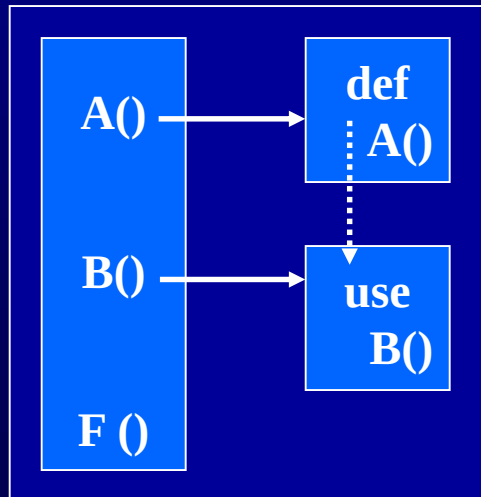
Types of Def-Use Pairs



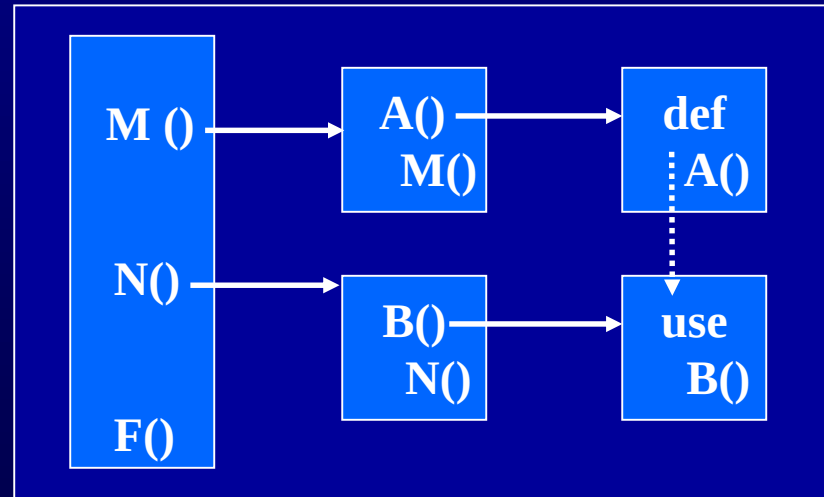
intra-procedural data flow
(within the same unit)



inter-procedural
data flow



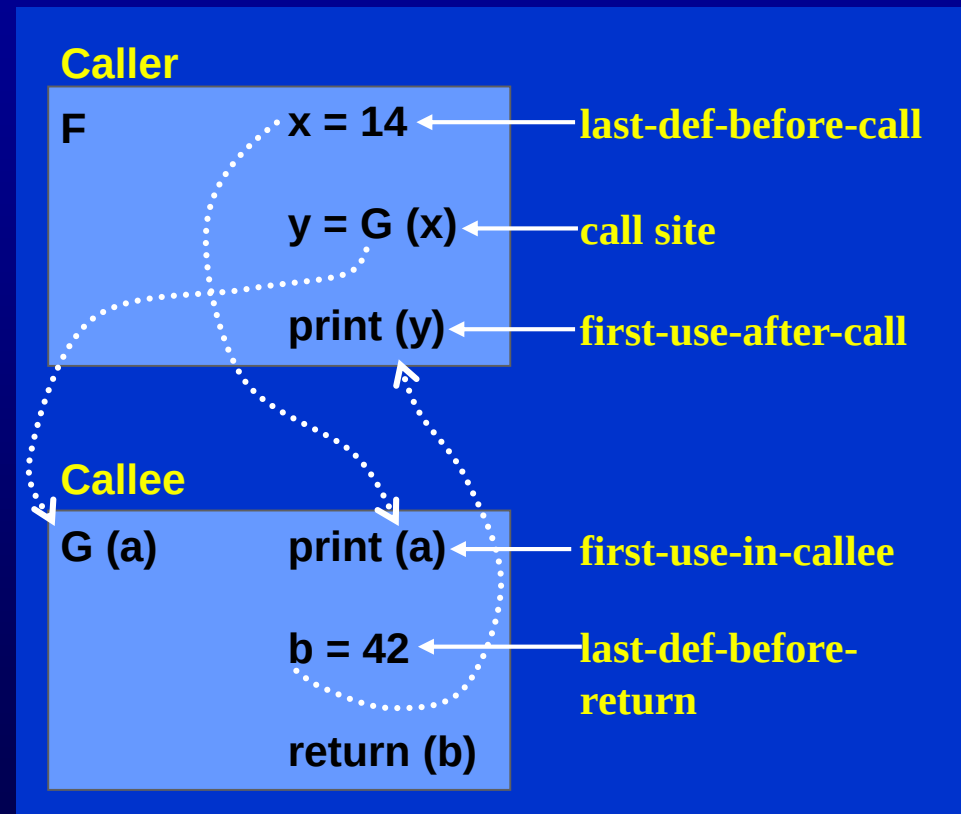
object-oriented direct
coupling data flow



object-oriented indirect
coupling data flow

Coupling-Based Testing (from Ch 2)

- Test data and control **connections**
- **Derived** from previous work for procedural programs
- Based on insight that integration occurs through **couplings** among software artifacts



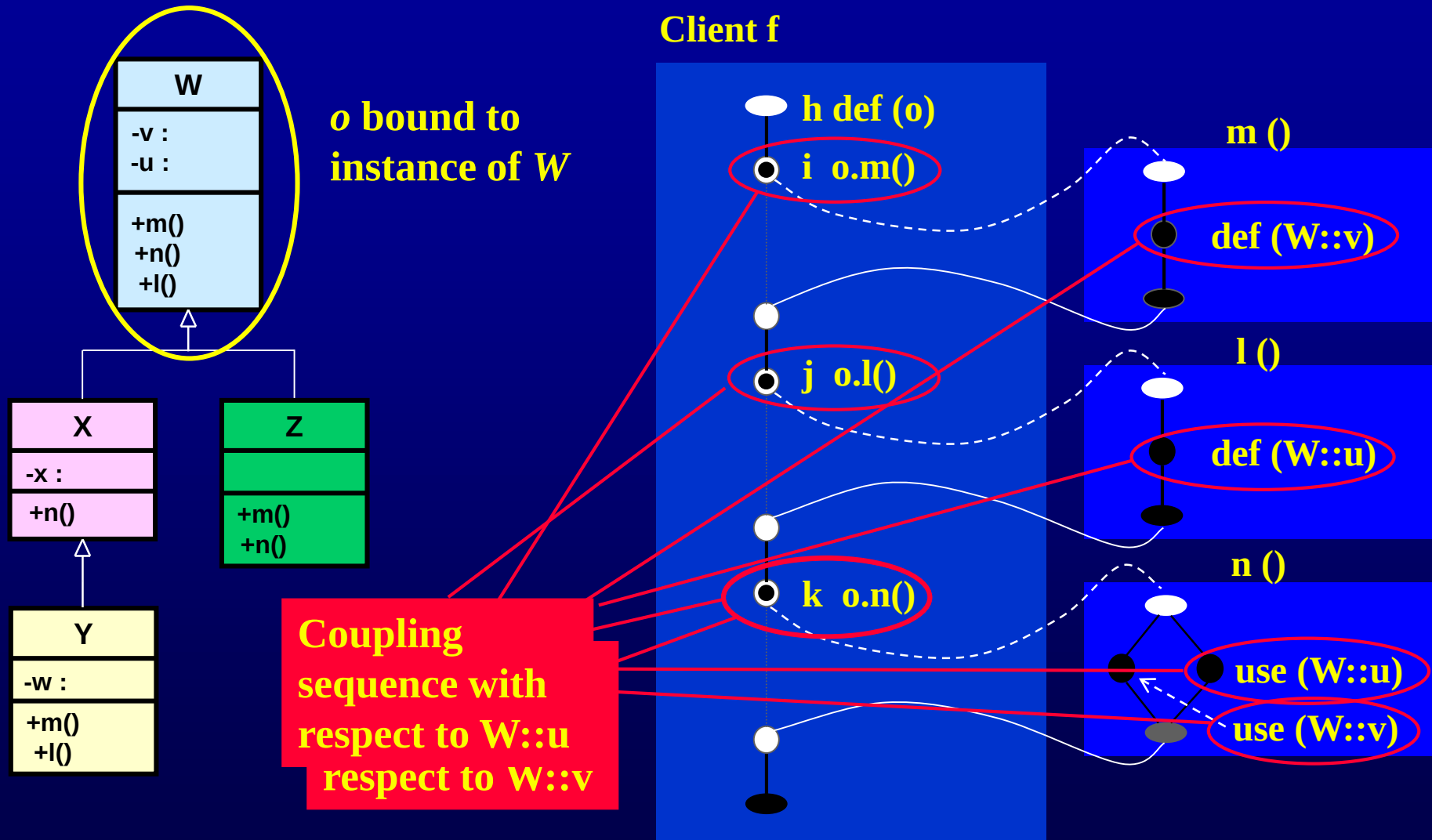
Polymorphic Call Set

Set of methods that can **potentially** execute as result of a method call through a particular instance context

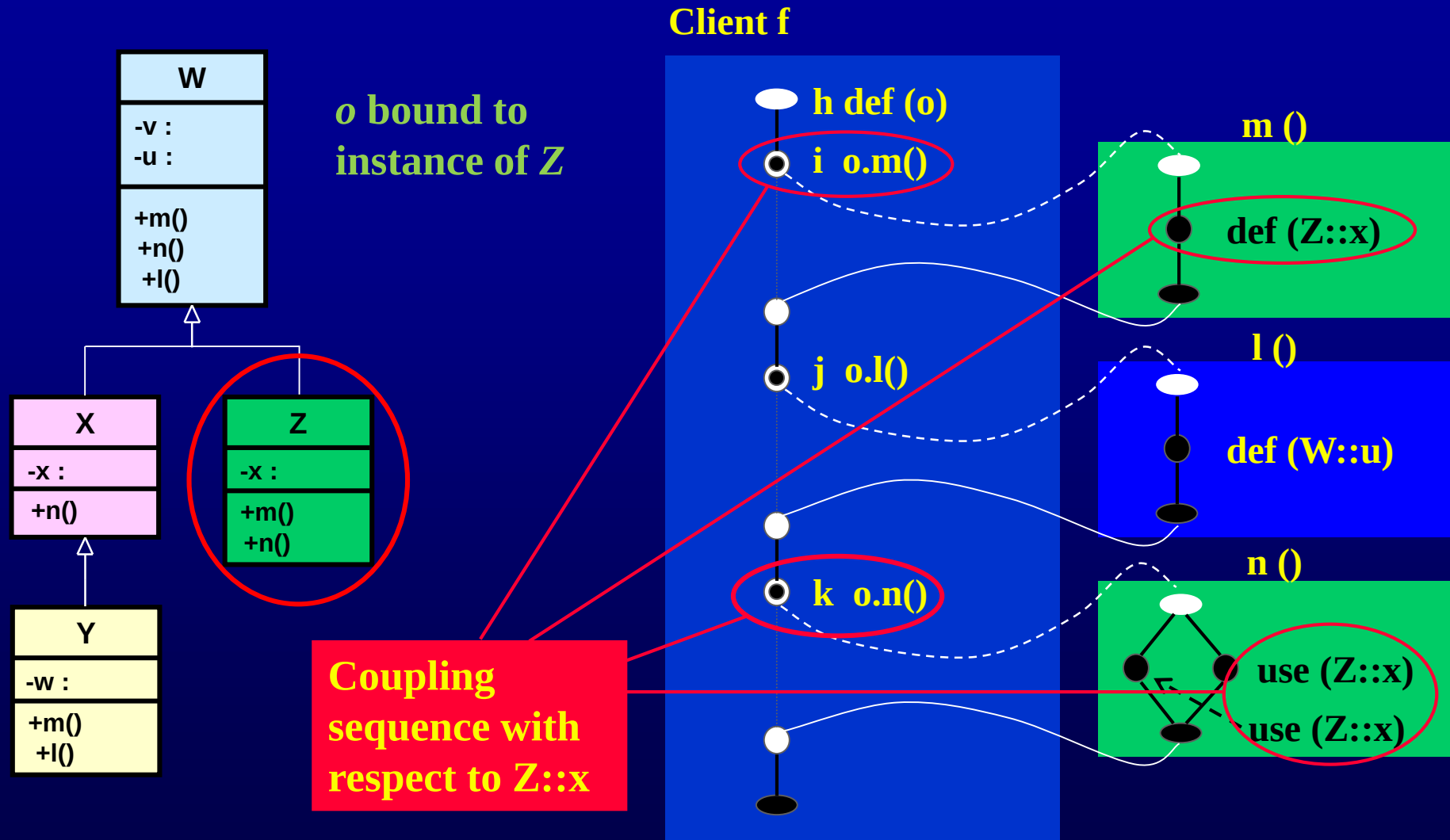
$pcs(o.m) = \{W::m, Y::m, X::m\}$

```
public void f ( W o )  
{  
    ...  
j    o.m();  
    ...  
l    o.l();  
    ...  
k    o.n();  
}
```

Example Coupling Sequence



Example Coupling Sequence (2)



Section 7.1 Outline

1. Overview
2. Types of Object-Oriented Faults
 1. Example
 2. The Yo-Yo Graph and Polymorphism
 3. Categories of Inheritance Faults
 4. Testing Inheritance, Polymorphism and Dynamic Binding
 5. **Object-Oriented Testing Criteria**

Testing Goals

- We want to test how a method can **interact** with instance bound to object ***o***:
 - Interactions occur through the **coupling sequences**
- Need to consider the **set of interactions** that can occur:
 - What **types** can be bound to ***o***?
 - Which **methods** can actually execute? (polymorphic call sets)
- Test **all couplings** with **all type** bindings possible

All-Coupling-Sequences

All-Coupling-Sequences (ACS) : For every coupling sequence S_j in $f()$, there is at least one test case t such that there is a coupling path induced by $S_{j,k}$ that is a sub-path of the execution trace of $f(t)$

- At least one **coupling path** must be executed
- Does **not** consider inheritance and polymorphism

All-Poly-Classes

All-Poly-Classes (APC) : For every coupling sequence $S_{j,k}$ in method $f()$, and for every class in the family of types defined by the context of $S_{j,k}$, there is at least one test case t such that when $f()$ is executed using t , there is a path p in the set of coupling paths of $S_{j,k}$ that is a sub-path of the execution trace of $f(t)$

- Includes **instance contexts** of calls
- At least one test for **every type** the object can bind to
- Test with every possible **type substitution**

All-Coupling-Defs-Uses

All-Coupling-Defs-Uses (ACDU) : For every coupling variable v in each coupling $S_{j,k}$ of t , there is a coupling path induced by $S_{j,k}$ such that p is a sub-path of the execution trace of $f(t)$ for at least one test case t

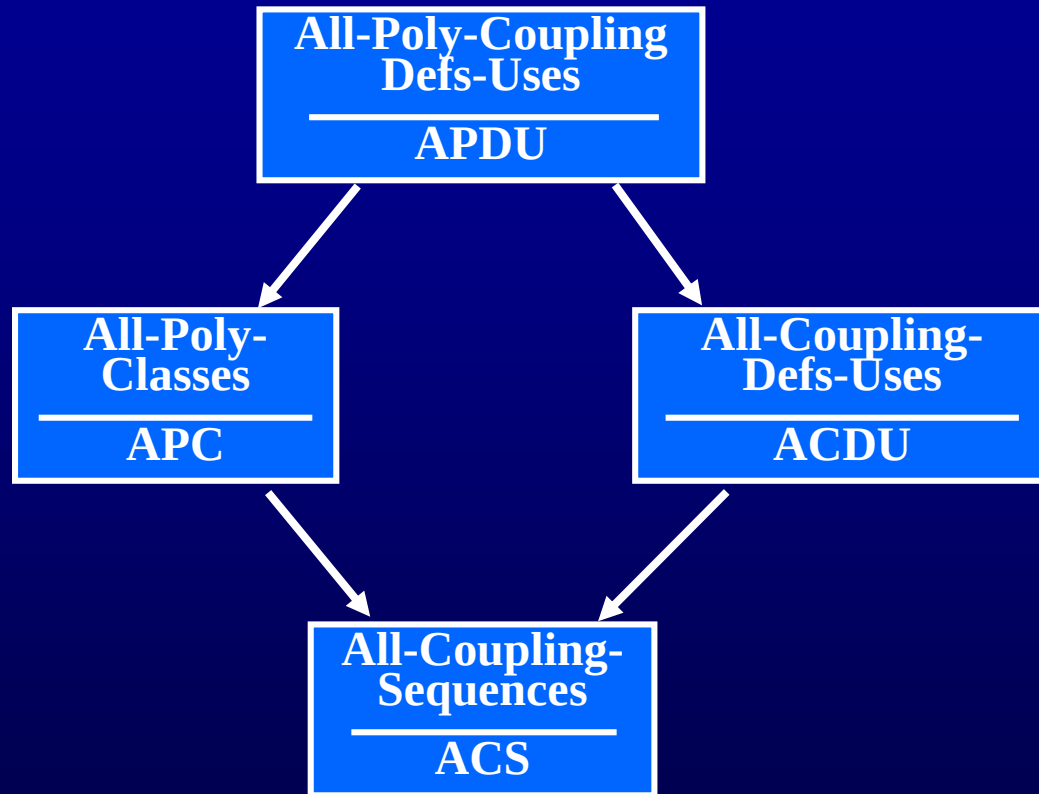
- Every **last definition** of a coupling variable reaches every first use
- Does **not** consider inheritance and polymorphism

All-Poly-Coupling-Defs-and-Uses

All-Poly-Coupling-Defs-and-Uses (APDU) : For every coupling sequence $S_{j,k}$ in $f()$, for every class in the family of types defined by the context of $S_{j,k}$, for every coupling variable v of $S_{j,k}$, for every node m that has a last definition of v and every node n that has a first-use of v , there is at least one test case t such that when $f()$ is executed using t , there is a path p in the coupling paths of $S_{j,k}$ that is a sub-path of the trace of $f()$

- Every **last definition** of a coupling variable reaches every first use for **every type binding**
- **Combines** previous criteria
- Handles inheritance and polymorphism
- Takes definitions and uses of variables into account

OO Coverage Criteria Subsumption



Conclusions

- A **model for understanding** and analyzing faults that occur as a result of inheritance and polymorphism
 - Yo-yo graph
 - Defs and Uses of state variables
 - Polymorphic call set
- Technique for **identifying data flow anomalies** in class hierarchies
- A **fault model** and specific faults that are common in OO software
- Specific **test criteria** for detecting such faults

Introduction to Software Testing

Chapter 7.2

Engineering Criteria for Technologies

Paul Ammann & Jeff Offutt

<http://www.cs.gmu.edu/~offutt/softwaretest/>

Chapter 7 Outline

1. Object-Oriented Software
2. **Web Applications and Web Services**
3. Graphical User Interfaces
4. Real-Time and Embedded Software

Section 7.2 Outline

Most of these ideas were developed after 2000
Few are widely used
Most adapt graph-based testing from Chapter 2

1. **Overview**
2. **Static Hyper Text Web Sites**
3. **Dynamic Web Applications**
 1. Client-side testing
 2. Server-side testing
4. **Web Services**

Issues in Testing Web Software

- A **web application** is a program that is **deployed** on the web
 - Usually uses **HTML** as the user interface
 - Web-deployment means they are **available worldwide**
 - They accept **requests** through HTTP and return **responses**
 - HTTP is **stateless** – each request/response pair is independent
- Web applications are usually very **competitive**
- A **web service** is a web-deployed program that accepts **XML** messages wrapped in **SOAP**
 - Usually **no UI** with humans
 - Service must be **published** so other services and applications can **discover** them

Web Software

- Composed of **independent, loosely coupled** software components
 - All communication is through **messages**
 - Web application messages always go through **clients**
 - The only **shared memory** is through the session object – which is very restricted
 - The definition of **state** is quite different
- Inherently **concurrent** and often **distributed**
- Most components are relatively **small**
- Uses numerous new **technologies**, often mixed together

Deploying Software

- **Bundled** : Pre-installed on computer
- **Shrink-wrap** : Bought and installed by end-users
- **Contract** : Purchaser pays developer to develop and install, usually for a fixed price
- **Embedded** : Installed on a hardware device, usually with no direct communication with user
- **Web : Executed across the Internet through HTTP**

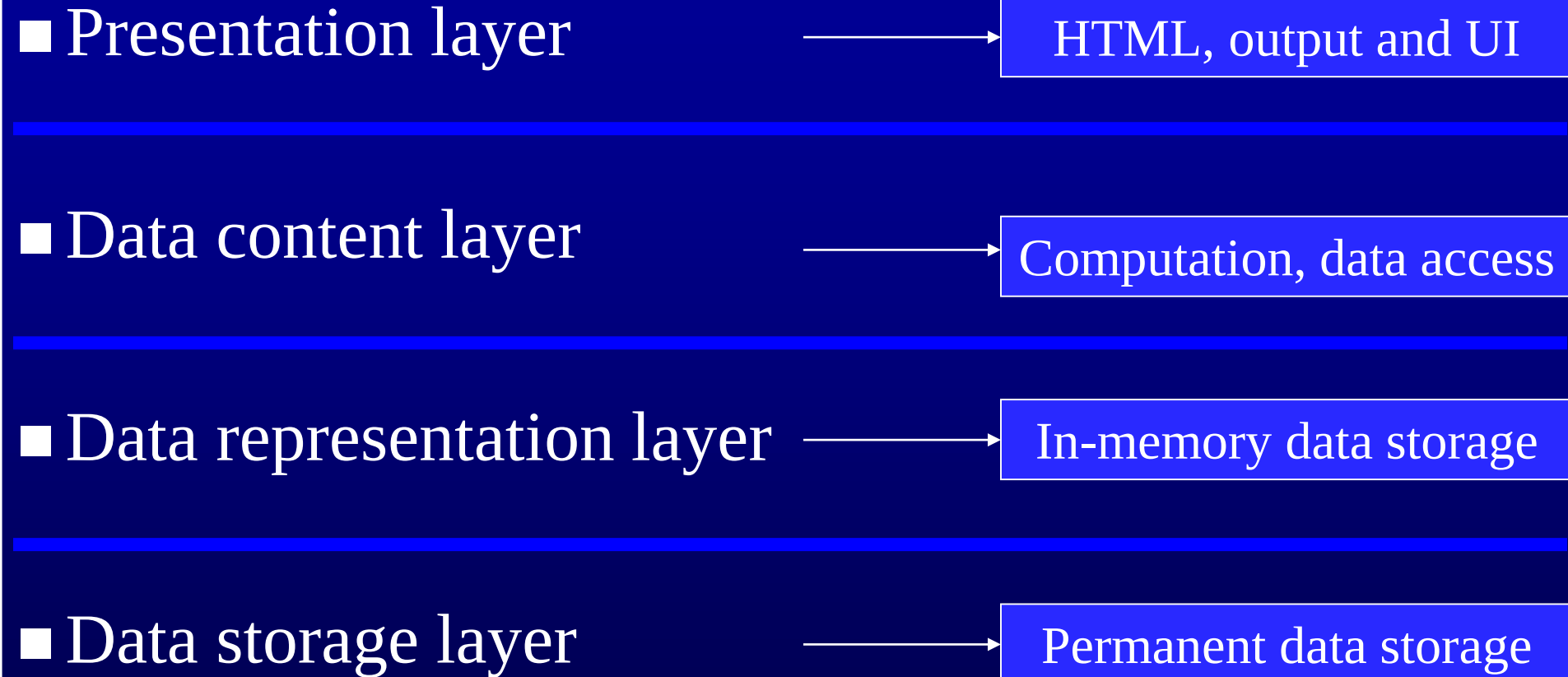
General Problem

- Web applications are **heterogeneous, dynamic** and must satisfy very high **quality attributes**
- Use of the Web is hindered by **low quality** Web sites and applications
- Web applications need to be **built** better and **tested** more

Problem Parameters

- HTTP is a **stateless protocol**
 - Each request is **independent** of previous request
- Servers have little information about **where** a request comes from
- Web site software is **extremely loosely coupled**
 - Coupled through the Internet – separated by **space**
 - Coupled to **diverse** hardware devices
 - Written in **diverse** software languages

Separation of Concerns in Web Apps



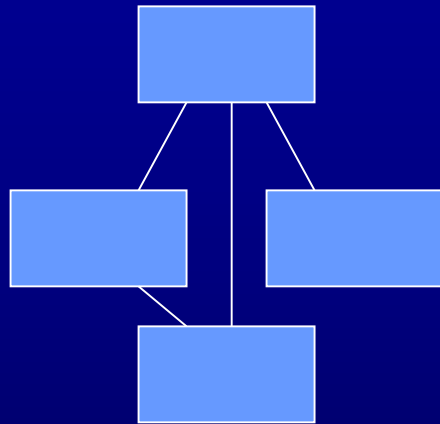
Differences in Testing Web Software

- Traditional graphs do **not apply**
 - Control flow graph
 - Call graph
- **State behavior** is hard to model and describe
- All **inputs** go through the HTML UI – low **controllability**
- Hard to get access to server-side **state** (memory, files, database) – low **observability**
- Not clear what **logic predicates** can be effectively used
- No model for **mutation operators** on web software

New Essential Problems of Web Apps

1. Web site applications feature **distributed integration** and are **extremely loosely coupled**
 - Internet and diverse hardware / software
2. HTML forms are **created dynamically** by web applications
 - UI created on demand and can vary by user and time
3. Users can **change the flow of control** arbitrarily
 - back button, forward button, URL rewriting, refresh
4. **Dynamic integration** of new software components
 - new components can be added during execution

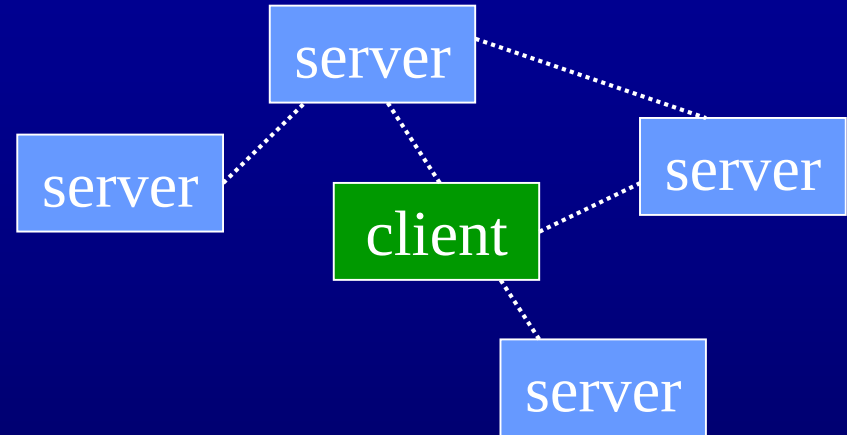
Problem 1: Loosely Coupled



Traditional software

Connected by calls and message passing

High and moderate coupling



Web-based software

Connected with HTTP and XML

Loose, *extremely* loose, distributed coupling

How can we ensure the reliability of this type of software?

Extremely Loose Coupling

- ***Tight Coupling*** : Dependencies among the methods are encoded in their **logic**
 - Changes in A may require changing logic in B
- ***Loose Coupling*** : Dependencies among the methods are encoded in the **structure** and **data flows**
 - Changes in A may require changing data uses in B
- ***Extremely Loose Coupling (ELC)*** : Dependencies are encoded only in the data **contents**
 - Changes in A only affects the contents of B's data

Problem 2: Dynamic Flow of Control

WebPics
How you'ns doin' Jeff Offutt!

[Search](#)

Recommended Movies

X

XX

XXX

[Examine queue](#) *(Warning: Queue empty)*

[View account](#)

WebPics
Huan ying guang ling, Li Nan!

[Search](#)

Recommended Movies

A

B

C

D

[Examine queue](#)

[View account](#)

[Frequent customer bonus](#)

How can we ensure the reliability of this type of system?

Dynamic Execution of Web Apps

- Parts of the program are **generated dynamically**
- **Dynamic web pages** are created when users make requests
- Different users will see **different programs !**
- The potential control, ala the traditional control flow graph, **cannot be known ahead of time**

The potential flow of control cannot be known statically

Problem 3: User Control Flow

- Users can make **unexpected changes** to the flow of control
 - *Operational transitions* are NOT based on an HTML link or program statement
 - Back button, forward button, refreshing, caching, URL rewriting
- Web software platforms offer **new control flow** mechanisms
 - Forward, redirect, asynchronous message passing (Ajax)
- **State** is stored in the server and in the HTML in the client's browser
- These transitions can cause **unanticipated changes** to the state of the web application

How can we ensure the reliability of this type of software?

Problem 4: Dynamic Integration

- Software modules can **dynamically** integrate with others if they use the same data structures
- **EJBs** can be inserted into web applications, which can immediately start using them
- Web services **find and bind** to other web services dynamically

Section 7.2 Outline

1. Overview
2. **Static Hyper Text Web Sites**
3. Dynamic Web Applications
 1. Client-side testing
 2. Server-side testing
4. Testing Web Services

Testing Static Hyper Text Web Sites

- This is **not** program testing, but checking that all the HTML **connections** are valid
- The main issue to test for is **dead links**
- We should also evaluate
 - **Load** testing
 - **Performance** evaluation
 - **Access control** issues
- The usual model is that of a graph
 - **Nodes** are web pages
 - **Edges** are HTML links

Section 7.2 Outline

1. Overview
2. Static Hyper Text Web Sites
3. **Dynamic Web Applications**
 1. Client-side testing
 2. Server-side testing
4. Testing Web Services

Testing Dynamic Hyper Text Web Sites

- The **user interface** is on the client
- Some **software** is on the client (scripts such as Javascript)
- Most **software** is on the **server**
- **Client-side testing** does not access source or state on the server
- **Server-side testing** can use the source or the server state

Section 7.2 Outline

1. Overview
2. Testing Static Hyper Text Web Sites
3. Testing Dynamic Web Applications
 1. **Client-side testing**
 2. Server-side testing
4. Testing Web Services

Client-Side (Black-Box) Testing

- The UI and the software are on **separate** computers
- The inputs to web software are defined by the **HTML form** elements
 - Text boxes, buttons, dropdown lists, links, etc
- Techniques for **generating** values
 - Supplied by the tester
 - Generated randomly
 - User session data – data collected from previous users of the software
- **Choosing** values
 - Bypass testing – values that violate constraints on the inputs, as defined by client-side information
- The problem of **finding all screens** in a web application is undecidable

Test Value Selection

■ Challenge

- How to automatically provide effective test values ?

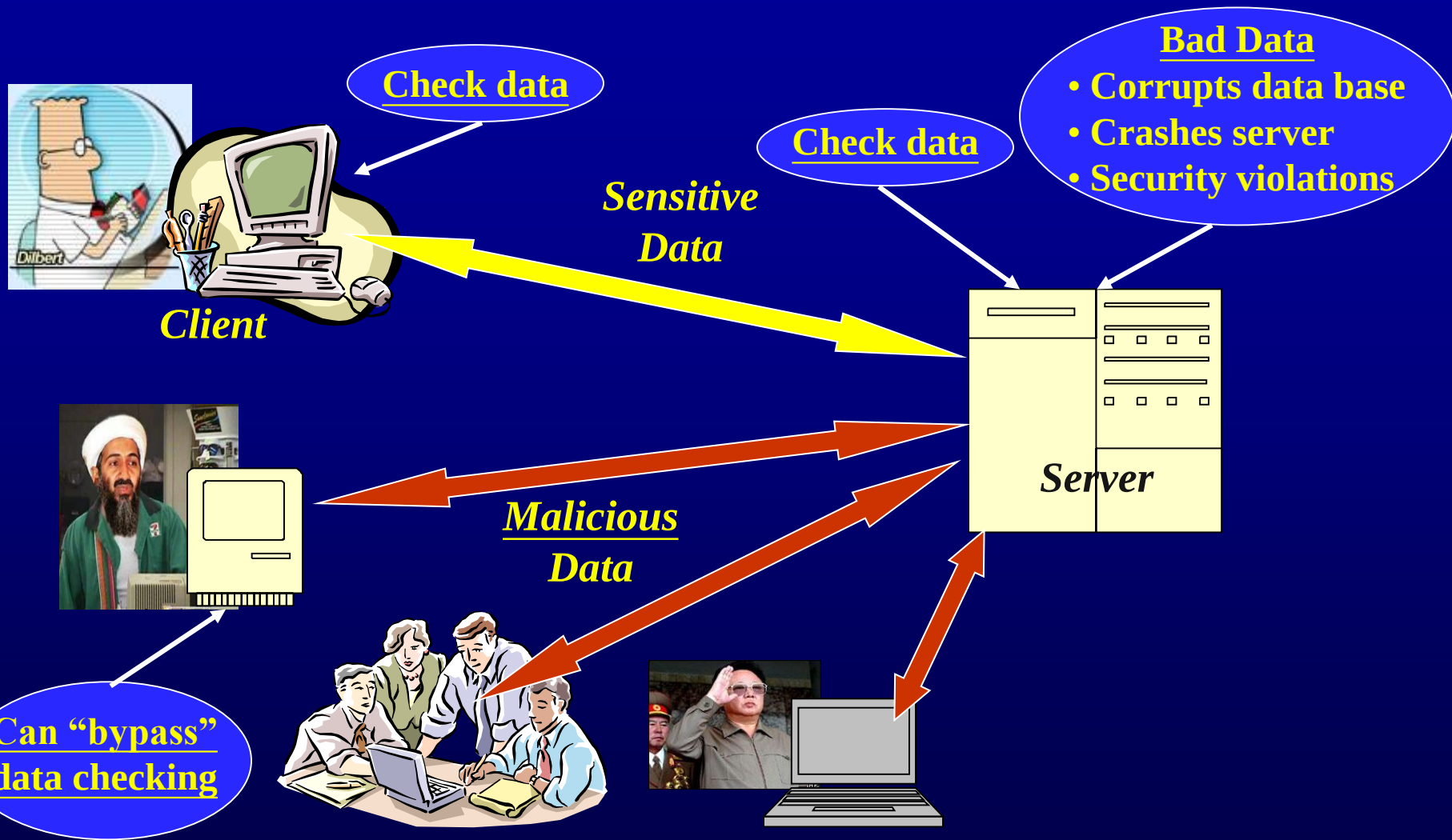
■ *Semantic Domain Problem* (SDP)

- Values within the application domain are needed
- Enumeration of all possible test values is inefficient

■ Possible solutions

- **Random** values (ineffective)
- **Automatically** generated values (very hard!)
- **User** data (incomplete)
- Study application and **construct** a set of values (feasible)
- **Tester-supplied** inputs (feasible but expensive)

Web Application Input Validation



Bypass Testing

- “*bypass*” client-side **constraint enforcement**
- Bypass testing constructs tests to intentionally **violate constraints** :
 - Eases **test automation**
 - Validates **input validation**
 - Checks **robustness**
 - Evaluates **security**

User Name: Age:

Version to purchase:

Small

Medium

Large

\$150

\$250

\$500



Invalid data, please correct ...

User Name: Alan<Turing

Age: 500

Username should be plain text only.

Age should be between 18 and 150.

Version to purchase:

Small

Medium

Large

\$150

\$250

\$500



Abbreviated HTML

<FORM >

<INPUT Type="text" Name="username" Size=20>

<INPUT Type="text" Name="age" Size=3 Maxlength=3>

<P> Version to purchase:

...

<INPUT Type="radio" Name="version" Value="150" Checked>

<INPUT Type="radio" Name="version" Value="250">

<INPUT Type="radio" Name="version" Value="500">

<INPUT Type="submit" onClick="return checkInfo(this.form)">

<INPUT Type="hidden" isLoggedIn="no">

</FORM>

Bypass Behavior

- Extremely loose **coupling** ...
- combined with the **stateless** protocol ...
- allows users to easily **bypass** client-side checking :

Users can save and modify the HTML

Saved & Modified HTML

<FORM >

<INPUT Type="text" Name="username" Size=20>

<INPUT Type="text" Name="age" Size=3 ~~Maxlength=3~~>

<P> Version to purchase:

...

<INPUT Type="radio" Name=

<INPUT Type="radio" Name= version value= 250 >

<INPUT Type="radio" Name="version" Value="25" Checked>

<INPUT Type="submit" ~~onClick="return checkInfo (this.form)"~~>

<INPUT Type="hidden" isLoggedIn= yes >

Allows an input with arbitrary age,
no checking, cost=\$25 ...

'<' can crash an XML parser

Text fields can have SQL statements

</FORM>

SQL Injection

User Name: turing ' OR '1'='1 **Password:** enigma ' OR '1'='1

Original SQL:

```
SELECT username FROM adminuser WHERE  
username='turing' AND password ='enigma'
```

“injected”

SQL:

```
SELECT username FROM adminuser WHERE username='turing'  
OR '1' = '1' AND password ='enigma' OR '1' = '1'
```


Bypass Testing

- This example illustrates how users can “bypass” client-side **constraint enforcement**
- Bypass testing constructs tests to **intentionally violate constraints**
 - Eases test **automation**
 - Checks **robustness**
 - Evaluates **security**
- Preliminary results
 - Rules for constructing tests
 - Successfully found errors in numerous Web apps

Applying Bypass Testing

Validating input data on the client is like asking your opponent to hold your shield in a sword fight

- **Analyze** HTML to extract each form element
- **Model** constraints imposed by HTML and JavaScript
- **Rules** for data generation :
 - From **client-side** constraints
 - Typical **security** violations
 - **Common** input mistakes

Types of Client Input Validation

- Client side **input validation** is performed by HTML form controls, their attributes, and client side scripts that access DOM
- **Validation types** are categorized as HTML and scripting
 - HTML supports **syntactic** validation
 - Client scripting can perform both syntactic and **semantic** validation

HTML Constraints

- Length (*max input characters*)
- Value (*preset values*)
- Transfer Mode (*GET or POST*)
- Field Element (*preset fields*)
- Target URL (*links with values*)

Scripting Constraints

- Data Type (*e.g. integer check*)
- Data Format (*e.g. ZIP code format*)
- Data Value (*e.g. age value range*)
- Inter-Value (*e.g. credit # + exp. date*)
- Invalid Characters (*e.g. <,../,&*)

Example Client-Side Constraint Rules

- Violate **size restrictions** on strings
- Introduce values **not included** in static choices
 - **Radio** boxes
 - Select (**drop-down**) lists
- Violate **hard-coded values**
- Use values that JavaScripts flag as **errors**
- Change “**transfer mode**” (get, post, ...)
- Change destination **URLs**

Example Server-Side Constraint Rules

- Data **type** conversion
- Data **format** validation
- **Inter-field** constraint validation
- **Inter-request** data fields (cookies, hidden)

Example Security Violation Rules

Potential Illegal Character	Symbol
Empty String	
Commas	,
Single and double quotes	' or ''
Tag symbols	Tag symbols < and >
Directory paths	/
Strings starting with forward slash	/
Strings starting with a period	.
Ampersands	&
Control character	NIL, newline
Characters with high bit set	254 and 255
Script symbols	<javascript> or <vbscript>

Test Value Selection

■ Challenge:

- How to **automatically** provide effective test values?

■ *Semantic Domain Problem* (SDP)

- Values within the **application domain** are needed
- Enumeration of **all possible** test values is inefficient

■ Possible Solutions

- **Random** Values (ineffective – lots of **junk**)
- **Automatically** generated values (very **hard**)
- Taking values from **session log** files (feasible but **incomplete**)
- **Tester** input (**feasible**)

■ Our tool used an **input domain** created by parsing the interface and tester input

Real-World Examples

atutor.ca
Atalker

nytimes.com
Us-markets

bankofamerica.com
ATM locator, Site search

demo.joomla.or
Poll, Users

mutex.gmu.edu
Login form

comcast.com
Service availability

phpMyAdmin
Main page,
Set Theme,
SQL Query,
DB Stats

yahoo.com
Notepad, Composer,
Search reminder,
Weather Search

ecost.com
Detail submit,
Shopping cart control

brainbench.com
Submit Request
Info, New user

barnesandnoble.com

google.com
Freogle, Language tools

myspace.com
Events & Music
Search

amazon.com
Cart manager
Book search/results

pages.mkes.com
Registration

amazon.com

Handle buy

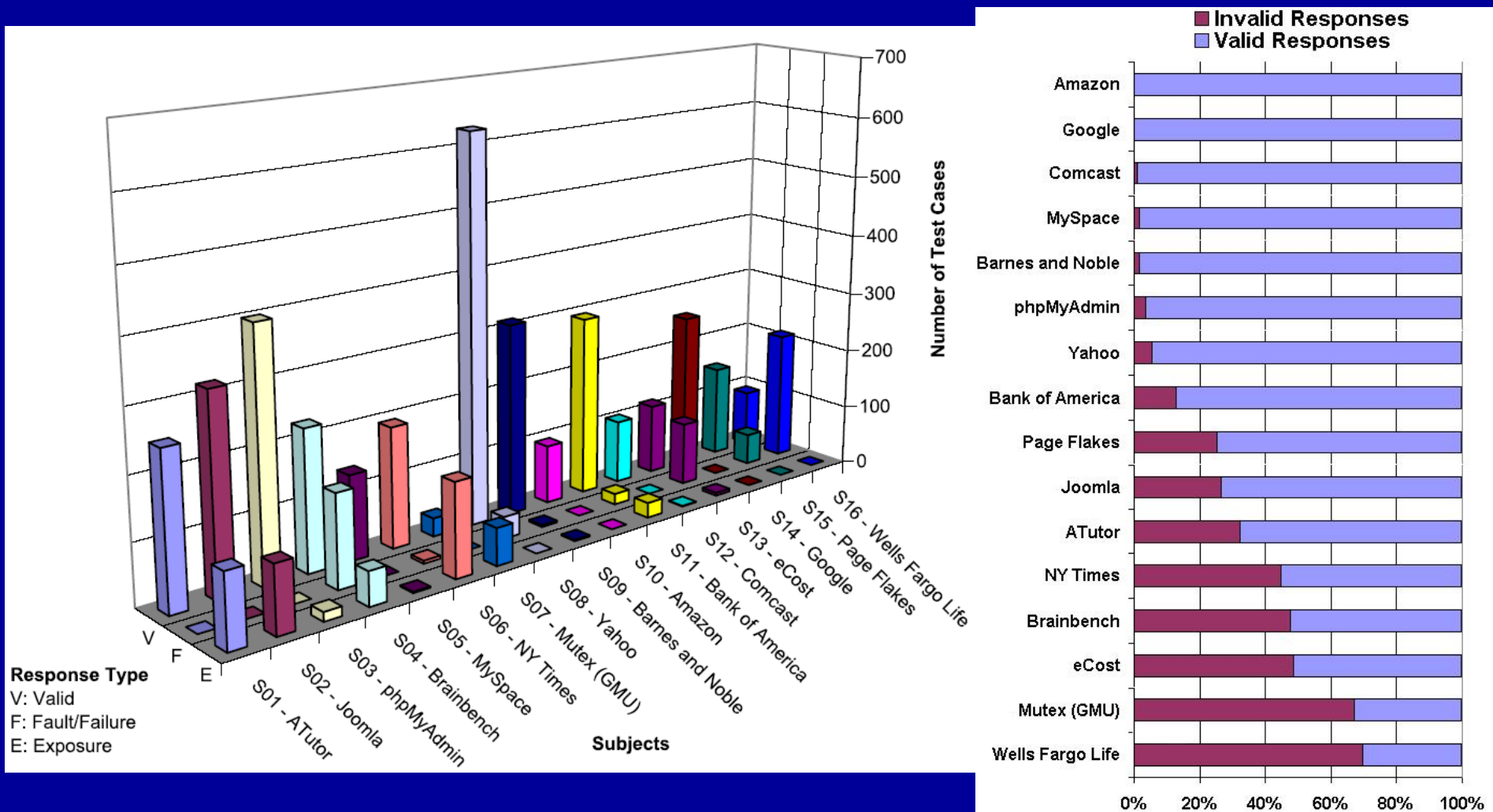
Quote search

Pure black-box testing
means
no source (or permission) needed!

Output Checking

- **(V) Valid Responses** : invalid inputs are adequately processed by the server
 - (V1) Server acknowledges the invalid request and provides an explicit message regarding the violation
 - (V2) Server produces a generic error message
 - (V3) Server apparently ignores the invalid request and produces an appropriate response
 - (V4) Server apparently ignores the request completely
- **(F) Faults & Failures** : invalid inputs that cause abnormal server behavior (*typically caught by web server when application fails to handle the error*)
- **(E) Exposure** : invalid input is not recognized by the server and abnormal software behavior is exposed to the users
- These do not capture whether the valid responses **corrupted data** on the server

Results



Section 7.2 Outline

1. Overview
2. Testing Static Hyper Text Web Sites
3. Testing Dynamic Web Applications
 1. Client-side testing
 2. **Server-side testing**
4. Testing Web Services

Server-Side (White-Box) Testing

- If we have access to the source on the server, we can try to **model** the web software
- Many testing criteria on non-web software rely on a static **control flow graph**
 - Edge testing, data flow, logic coverage, ...
 - Also slicing, change impact analysis, ...
- The standard control flow graph **cannot** be computed for web applications !
- **But all the pieces of the web pages and the programs are contained in the software presentation layer ...**

Atomic Sections :

HTML with *static structure* and *content variables*

	PrintWriter out = response.getWriter();
P1 =	out.println("<HTML>") out.println("<HEAD><TITLE>" + title + "</TITLE></HEAD>") out.println("<BODY>")
	if (isUser) {
P2 =	out.println(" <CENTER> Welcome!</CENTER>");
	for (int i=0; i<myVector.size(); i++) if (myVector.elementAt(i).size > 10)
P3 =	out.println(" " + myVector.elementAt(i) + " ");
	else
P4 =	out.println(" " + myVector.elementAt(i) + " ");
	} else
P5 =	{ }
P6 =	out.println("</BODY></HTML>");
	out.close ();

Atomic sections

Empty atomic section

Content variables

Atomic Sections

- *A section of HTML with the property that if any part of the section is sent to a client, the **entire section** is*
 - May include **JavaScript**
 - **All or nothing** property
- An **HTML file** is an atomic section
- **Content variable** : A program variable that provides data to an atomic section
- Atomic sections may be **empty**

Modeling Web Applications

- Restricted to the **presentation layer** only
- Two levels of **abstraction**
 1. **Component Interaction Model (CIM)**
 - Models individual components
 - Combines atomic sections
 - Intra-component
 2. **Application Transition Graph (ATG)**
 - Each node is one CIM
 - Edges are transitions among CIMs
 - Inter-component

Component Expressions

- Atomic sections are **combined** to model dynamically generated web pages
- Four ways to combine:
 1. Sequence : $p1 \bullet p2$
 2. Selection : $(p1 \mid p2)$
 3. Iteration : $p1^*$
 4. Aggregation : $p1 \{p2\}$
 - $p2$ is included inside of $p1$
- The previous example produces:
$$p \rightarrow p1 \bullet (p2 \bullet (p3 \mid p4)^* \mid p5) \bullet p6$$
- Composite sections can be produced automatically

Modeling Component Transitions

Five types of transitions

1. Simple Link Transition : An HTML link (<A> tag)
2. Form Link Transition : Form submission link
3. Component Expression Transition : Execution of a software component causes a component expression to be sent to the client
4. Operational Transition : A transition out of the software's control
 - Back button, Forward button, Refresh button, User edits the URL, Browser reloads from cache
5. Redirect Transition : Server side transition, invisible to user

Component Interaction Model : gradeServlet

	ID = request.getParameter ("Id"); passWord = request.getParameter ("Password"); retry = request.getParameter ("Retry"); PrintWriter out = response.getWriter();
P1 =	out.println ("<HTML> <HEAD><TITLE>" + title + "</TITLE></HEAD><BODY>")
	if ((Validate (ID, passWord)) {
P2 =	out.println (" Grade Report ");
	for (int I=0; I < numberOfCourse; I++)
P3 =	out.println(" " + courseName (I) + "" + courseGrade (I) + " ");
	} else if (retry < 3) {
P4 =	retry++; out.println ("Wrong ID or wrong password"); out.println ("<FORM Method=\"get\" Action=\"gradeServlet\">"); out.println ("<INPUT Type=\"text\" Name=\"Id\" Size=10>"); out.println ("<INPUT Type=\"password\" Name=\"Password\" Width=20>"); out.println ("<INPUT Type=\"hidden\" Name=\"Retry\" Value=" + (retry) + ">"); out.println ("<INPUT Type=\"submit\" Name=\"Submit\" Value=\"submit\">"); out.println ("Send mail to the professor");
	} else if (retry >= 3) {
P5 =	out.println ("<p>Wrong ID or password, retry limit reached. Good bye.") }
P6 =	out.println("</BODY></HTML>");

CIM for gradeServlet

- $S = \text{login.html}$
- $A = \{p_1, p_2, p_3, p_4, p_5, p_6\}$
- $CE = \text{gradeServlet} = p_1 \cdot ((p_2 \cdot p_3^*) \mid p_4 \mid p_5) \cdot p_6$
- $T = \{ \text{login.html} \longrightarrow \text{gradeServlet} [\text{get}, (\text{Id}, \text{Password}, \text{Retry})], \\ \text{gradeServlet}.p_4 \longrightarrow \text{sendMail} [\text{get}, ()], \\ \text{gradeServlet}.p_4 \longrightarrow \text{gradeServlet} [\text{get}, (\text{Retry})] \}$

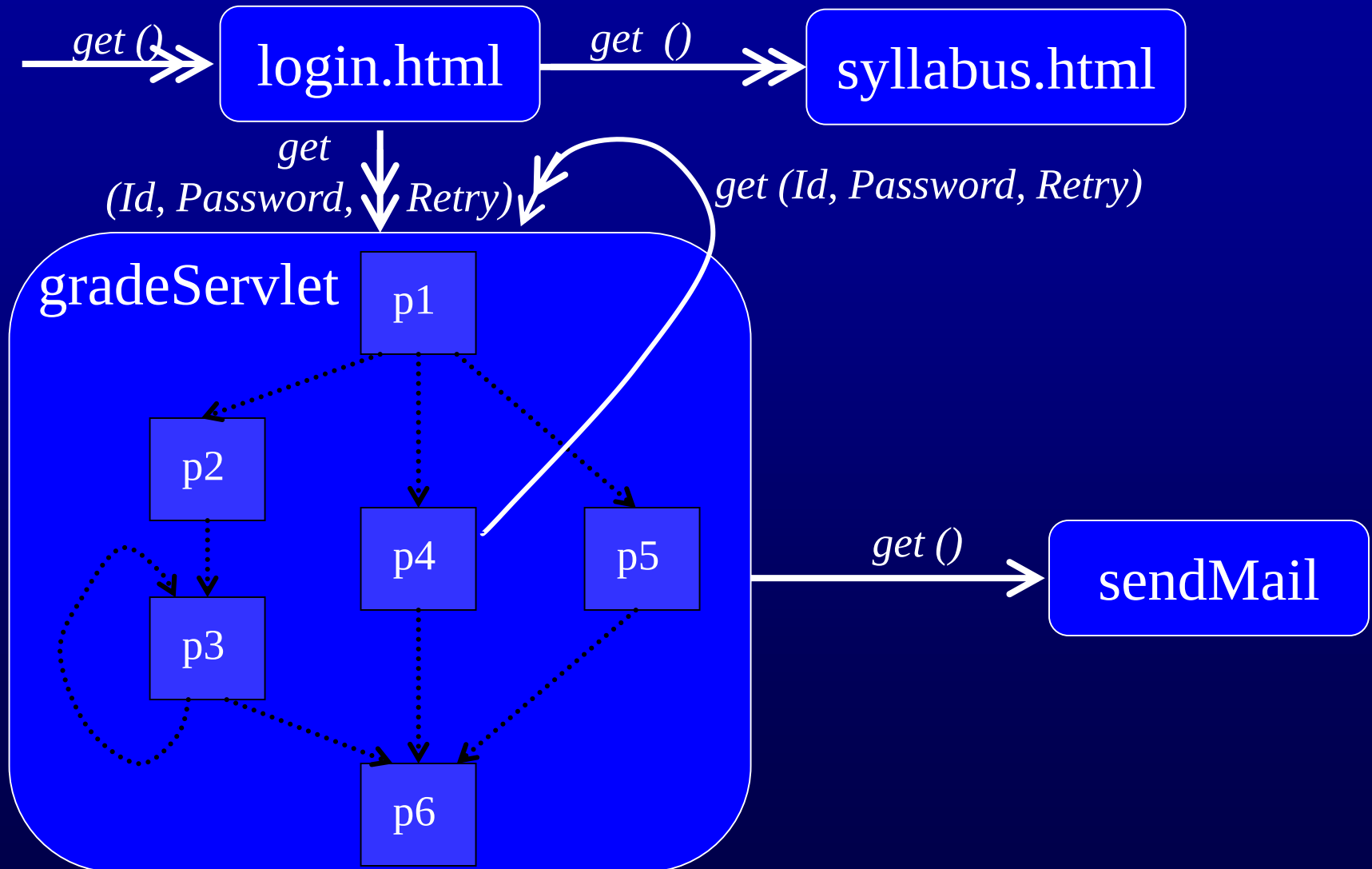
Application Transition Graph

- Γ : Finite set of web components
- Θ : Set of transitions among web software components
 - Includes type of HTTP request and data
- Σ : Set of variables that define the web application state
- α : Set of start pages

ATG for gradeServlet

- $\Gamma = \{ \text{login.html}, \text{gradeServlet}, \text{sendMail}, \text{syllabus.html} \}$
- $\Theta = \{ \text{login.html} \longrightarrow \text{syllabus.html} [\text{get}, ()],$
 $\text{login.html} \longrightarrow \text{gradeServlet} [\text{get}, (\text{Id}, \text{Password}, \text{Retry})],$
 $\text{gradeServlet.p}_4 \longrightarrow \text{sendMail} [\text{get}, ()],$
 $\text{gradeServlet.p}_4 \longrightarrow \text{gradeServlet} [\text{get}, (\text{Retry})] \}$
- $\Sigma = \{ \text{Id}, \text{Password}, \text{Retry} \}$
- $\alpha = \{ \text{login.html} \}$

ATG for gradeServlet



Atomic Section Modeling

- Atomic sections provide a **fundamental mechanism** to model Web applications presentation layer
- Can handle :
 - Distributed **integration**
 - **Dynamically** created HTML pages
 - **Operational** transitions
- Requires **deep analysis** of software source

Some Current Open Questions

■ How to define **data flow**?

- DU-pairs cannot be determined **statically** – uses cannot always be found

■ **Automatically** generating ATG

■ **Issues** not handled:

- Session data
- Multiple users
- Concurrency
- Input data
- Output validation
- Dynamic integration

Section 7.2 Outline

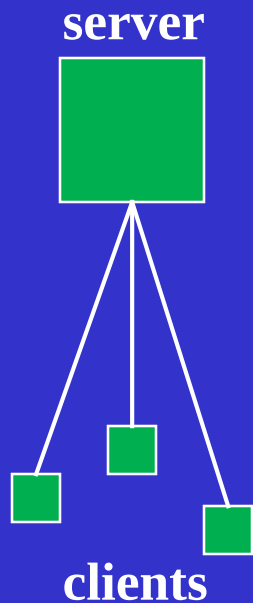
1. Overview
2. Static Hyper Text Web Sites
3. Dynamic Web Applications
 1. Client-side testing
 2. Server-side testing
4. **Testing Web Services**

Web Services

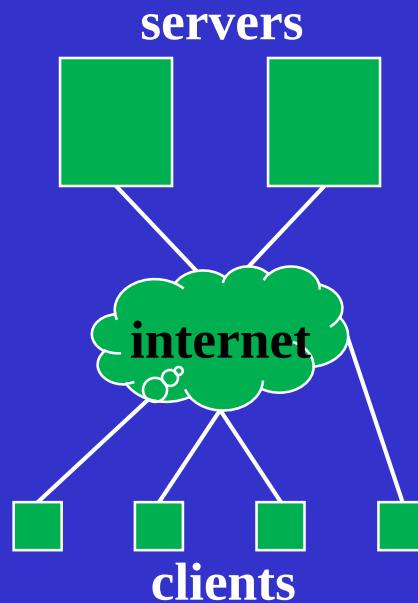
- A Web Service is a program that offers services over the Internet to other software programs
 - Internet-based
 - Uses SOAP and XML
 - Peer-to-peer communication
- Web service components can integrate dynamically, by finding other services during execution
- Web services transmit data that are formatted in XML

Web Service Architecture

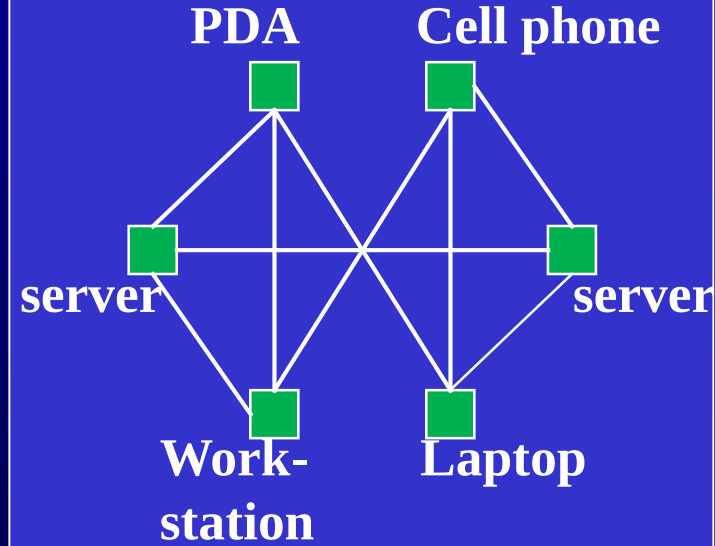
Client-server



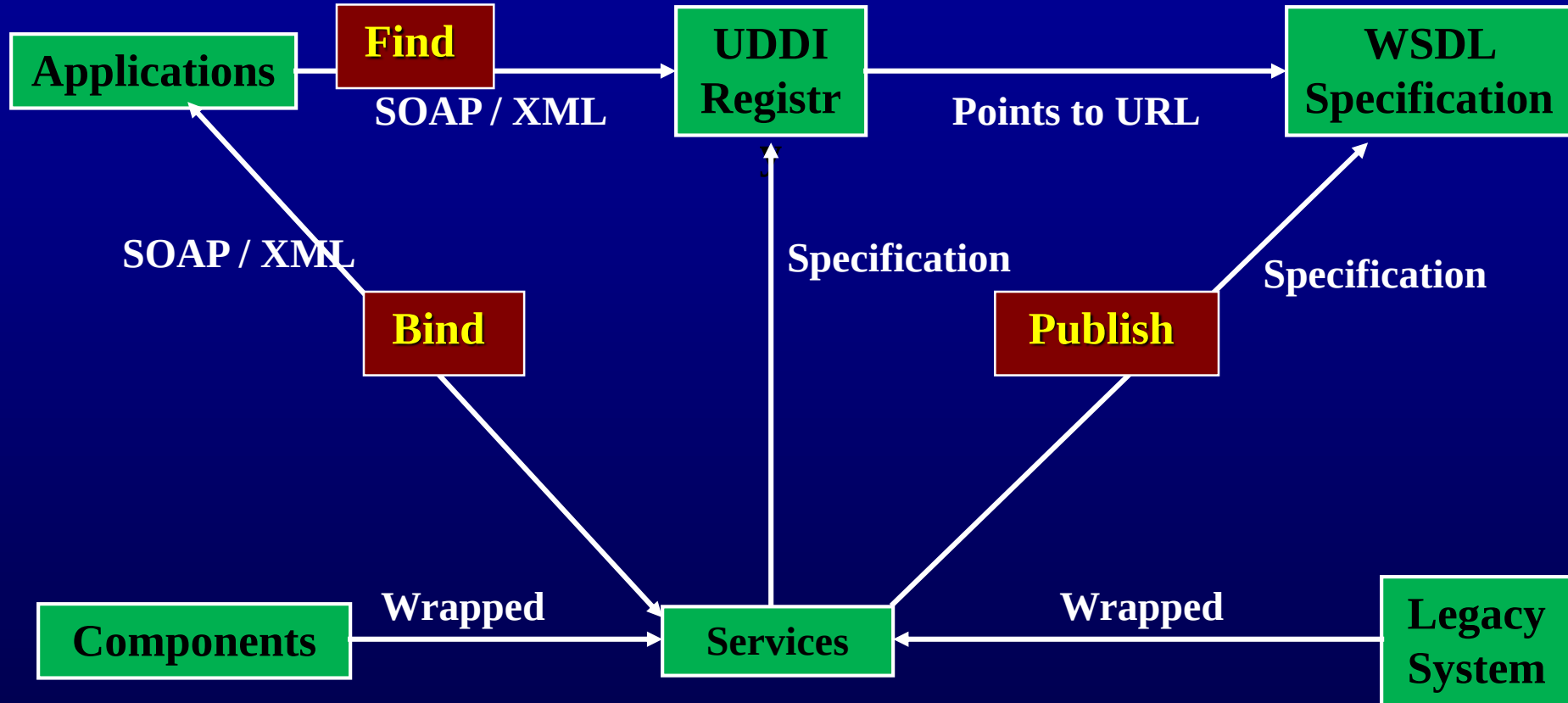
Web-based



Web Services



Web Service Technologies



Difficulties of Testing Web Services

- Web services are always **distributed**
- Most “peer-to-peer” communication is between services published by **different** organizations
 - **Trust** is a major issue holding back the adoption of web services !
- **Design** and **implementation** are almost never available
- **Structured messages** are transmitted
- Most testing research so far has focused on **messages**
 - **Syntax-based** test criteria have been proposed for Web services

Conclusions

- The Web provides a new way to **deploy** software
- Web applications:
 - offer many **advantages**
 - use many new technologies
 - introduce fascinating **new problems**
- **Web Software Engineering** is just beginning
- Two very useful techniques:
 - **Bypass testing** : Easy to automate – no source needed
 - **Atomic sections** : A fundamental model
- This is a very active **research** area